

To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior

Code:

```
geya@LenovoL480: ~/OS/Skil  ×  +  ▾
GNU nano 8.7.1 task4.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    char input[100];
    char name[50];
    char value[50];

    while (1) {

        printf("myshell> ");
        fgets(input, sizeof(input), stdin);

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        // Check for export command
        if (strncmp(input, "export ", 7) == 0) {

            if (sscanf(input + 7, "%49[^]=%49s", name, value) == 2) {
```

```
    if (name[0] >= 'A' && name[0] <= 'Z' ||  
        name[0] >= 'a' && name[0] <= 'z') {  
  
        setenv(name, value, 1);  
  
        printf("Variable %s set to %s\n", name, value);  
    }  
    else {  
        printf("Invalid variable name\n");  
    }  
}  
else {  
    printf("Invalid export syntax\n");  
}  
}  
  
// Display variable  
else if (strncmp(input, "print ", 6) == 0) {  
  
    char *result = getenv(input + 6);  
  
    if (result != NULL)  
        printf("%s\n", result);  
    else  
        printf("Variable not found\n");  
}
```

```

// Test child process
else if (strcmp(input, "child") == 0) {

    pid_t pid = fork();

    if (pid == 0) {
        printf("Child process sees:\n");

        char *result = getenv(name);

        if (result != NULL)
            printf("%s = %s\n", name, result);

        exit(0);
    }
    else {
        wait(NULL);
    }
}

else {
    printf("Invalid command\n");
}
}

```

```

    return 0;
}

```

Output:

```

geya@LenovoL480:~/OS/Skill/Week 5$ ./task4
myshell> export NAME= Geya
Variable NAME set to Geya
myshell> print NAME
Geya
myshell> child
Child process sees:
NAME = Geya
myshell> exit
geya@LenovoL480:~/OS/Skill/Week 5$ |

```